

Principle Violations

The `ItemController` correctly depends on the `IItemReader` abstraction (this is true for now, but, after adding the service layer, it will depend on `IGradeService`), and `ItemRepository` correctly implements it. However, inside `Program.cs`, the binding between `IItemReader` and `ItemRepository` is never registered in the DI container. This violates the Dependency Inversion Principle, because the composition root, `Program.cs`, is responsible for wiring abstractions to their concrete implementation. The fix is adding the Dependency Injection inside `Program.cs`.

`ItemRepository.cs` exposes protected fields and virtual methods, meaning it is designed to be extended through inheritance, which violates Open-Closed principle, because it invites subclasses to directly access and override the internal state (`_items`, `_nextId`). The Open-Closed principle states that a class should be open for extension, but closed for modification. The better way to achieve this is through a new interface implementation or composition. The fix for this is replacing “protected” with “private”, and removing the “virtual” keyword. The protected fields also violate the Encapsulation principle, which, even though it is not a SOLID principle, is a fundamental part of OOP.

Another issue is the absence of the service layer. Because of this, the controller (`ItemController.cs`) has multiple responsibilities: handling HTTP requests and also doing business logic. This breaks the Single Responsibility Principle. The fix for this is implementing the appropriate service which the Item Controller will use, moving the business logic to the service layer. Also, the `GetAll()` endpoint returns both the list of grades and the statistics, which also breaks the Single Responsibility Principle, the fix for this being adding a new endpoint, `api/grade/statistics`, and modifying `api/grade` to only return the list of grades.

Additional Improvements

The names of the classes, using the word `Item`, are too generic for this application. As such, I renamed them to use the word `Grade`: `GradeController.cs`, `IGradeReader.cs`, `Grade.cs` and `GradeRepository.cs`. I have also modified the `launchUrl` to `api/grade` in `launchSettings.json`, and renamed the appropriate occurrences of “item” in variable names to “grade”.

I kept the filtering for the `IsActive` attribute of the grades inside the repositories. I considered it as a flag that marks if a record was deleted or not, and, under this interpretation, it is more appropriate to filter it on the repository layer.